EXPENSE-RECONCILER · REVISIÓN EXPERTA

De lector de tickets a sistema que controla el gasto

Revisión experta de la API: qué ya tienes, qué te falta en compras y gastos, en qué orden construirlo — y el prompt listo para ejecutar el primer paso.

> **La conclusión en una frase:** hoy tu sistema defiende el cierre mensual del contador, pero no controla el gasto antes de que ocurra, y todavía no separa de verdad a una empresa de otra. Eso se corrige primero. Todo lo demás va después.

Qué contiene este documento

- **Veredicto y bases verificadas:** lo que ya funciona y no se toca.
- **P0, P1 y P2:** cada hueco con el problema, qué implementar y cómo saber que quedó bien.
- **Escáner y anti-duplicados:** el estado real, sin prometer lo que aún no se midió.
- **Anexo A:** el prompt completo, listo para copiar y pegar, para desarrollar el paso 1: roles y multi-empresa real.
- **Anexo B:** los comandos explicados en español sencillo, línea por línea, y el resumen para continuar si se acaban los tokens.

Para: Carlos
Proyecto: expense-reconciler-mvp — FastAPI, PostgreSQL, Ollama local
Fecha: 6 de noviembre de 2026
Complementa: el plan de negocio y de producto entregado el mismo día.

> **Cómo usar este documento:** lee el veredicto y el orden recomendado. Cuando quieras empezar, copia el prompt del Anexo A completo y pégalo en una sesión nueva. Ese prompt ya trae el contexto, las reglas y las pruebas. No necesitas explicarlo de nuevo.

1. Veredicto en 3 líneas

1. Tu API hoy es una herramienta de contador para defender un cierre, no una herramienta de empresa para controlar el gasto antes de que ocurra. Todo empieza cuando el ticket ya existe. Nada existe antes: no hay pedido, no hay aprobación previa, no hay presupuesto que frene.

2. Lo más grave no es una función que falta: es que **no hay roles ni multi-empresa real**. El propio código lo reconoce: está diseñado para «una máquina, un contador». Hoy, un usuario que ya entró al sistema puede ver datos de otra empresa si cambia el identificador de empresa en la petición.

3. Por eso el orden no es negociable: primero se cierra esa fuga y se separan los datos. Después viene el control de compras. Vender antes de cerrar la fuga es vender un problema.

> **Regla que sale de este veredicto:** la empresa de un usuario se toma siempre de su sesión, de su token, nunca de un dato que el cliente envíe en la petición. Si el cliente puede escribir a qué empresa pertenecen los datos que pide, la separación no existe.

2. Lo que ya tienes bueno, verificado

Esto se revisó sobre tu código, no se supuso. Consérvalo tal cual; es tu ventaja frente a un lector de tickets común:

- **Captura de tickets y facturas** con lectura OCR y visión solo cuando OCR no alcanza.
- **Puerta de confianza (*confidence gate*):** lo dudoso no pasa como bueno.
- **Cadena de custodia:** hash por archivo y documento que solo crece (*append-only*); nada se borra sin dejar rastro.
- **Muestra del 5 % y veredicto con intervalo de Wilson** para defender el cierre.
- **Conciliación ticket–banco** y **cierre mensual con PDF firmado**.
- **Inventario** alimentado desde las compras, **autorización por ticket**, **banco por CSV** y **exportación a Excel / CONTPAQI**.

No se listan rutas ni *endpoints* que no se hayan verificado en tu código. Si una función no aparece aquí, está en P0, P1 o P2, no dada por hecha.

P0 — Imprescindible antes de vender a empresas

P0 significa: sin esto, no se vende. Cada punto lleva el problema, qué implementar y cómo validar que quedó bien. Seis puntos en total, en esta página y las tres siguientes.

P0-1. Multi-empresa y roles de verdad

Problema. La empresa llega como parámetro de la petición. Quien ya entró puede pedir datos de otra empresa cambiando ese valor. Además no hay roles: todos pueden hacer todo.

Qué implementar.

- La empresa (`company_id`) se toma **siempre del token** del usuario que entró, jamás de un parámetro, del cuerpo de la petición o de una cabecera que el cliente controle.
- Toda consulta a la base de datos filtra por esa empresa, sin excepción.
- En PostgreSQL, seguridad a nivel de fila (*Row Level Security*) como segunda barrera: aunque una consulta olvide el filtro, la base no entrega filas de otra empresa.
- Roles: administrador, contador, aprobador, digitalizador y solo lectura. Cada rol puede hacer solo lo suyo.
- El ambiente de demostración vive en su propia empresa. Los datos demo jamás se mezclan con datos reales, ni siquiera para «probar rápido».

Cómo validar. Prueba de fuga cruzada obligatoria: el usuario A, de la empresa A, pide por su identificador exacto un ticket, una factura, un cierre y un reporte de la empresa B. Las cuatro respuestas deben ser «no encontrado» o «sin permiso», nunca los datos. Si una sola devuelve datos de B, el trabajo no está terminado.

> **Este es el paso que desarrolla el prompt del Anexo A.** Es el paso 1 de todo el plan. Ningún otro P0 se empieza antes de que la prueba de fuga pase.

P0-2. El CFDI como documento fiscal, no como un ticket más

Problema. Hoy del comprobante fiscal digital (CFDI) se leen datos como el identificador único fiscal (UUID), la serie, la moneda y el tipo de cambio, y después se pierden: no quedan guardados como parte del documento. Peor: la exportación a CONTPAQI llega a inventar el tipo de cambio cuando no lo tiene. Un dato inventado en un archivo contable es inaceptable.

Qué implementar.

- Guardar el CFDI completo como documento fiscal: UUID, serie y folio, RFC emisor y receptor, moneda, tipo de cambio real, subtotal, impuestos y total.
- El UUID es único por empresa: si ese UUID ya existe, es duplicado y no se registra otra vez (ver la regla anti-duplicados, página 10).
- Validar contra el Servicio de Administración Tributaria (SAT) si el CFDI está vigente o cancelado, y guardar el resultado con fecha de consulta.
- La exportación a CONTPAQI usa solo datos guardados. Si falta el tipo de cambio, el archivo marca el registro como incompleto; nunca lo inventa.

Cómo validar. Subir dos veces el mismo CFDI: la segunda debe aparecer en el listado de duplicados. Consultar un CFDI cancelado: debe quedar marcado como cancelado. Exportar un registro sin tipo de cambio: debe salir señalado como incompleto, sin ningún valor supuesto.

Por qué va segundo

Porque el contador vive del CFDI, no del ticket. La conciliación CFDI contra ticket es el gancho de venta número uno del plan de producto, y no se puede construir sobre datos que hoy se tiran.

P0-3. El gasto se controla antes: comparación triple

Problema. Hoy no existe orden de compra. Lo que el sistema llama «compras» es solo la entrada a inventario de un ticket que ya se pagó. La empresa se entera del gasto cuando ya ocurrió.

Qué implementar. El flujo completo, paso a paso:

1. **Requisición:** alguien pide lo que necesita, con motivo y centro de costo.
2. **Orden de compra:** la requisición aprobada se convierte en orden, con proveedor, cantidades y precios pactados.
3. **Recepción:** se registra qué llegó de verdad y en qué estado.
4. **Comparación triple (*3-way match*):** orden contra recepción contra factura o CFDI. Si las tres coinciden, el gasto pasa. Si algo difiere —cantidad, precio o total— queda marcado como discrepancia y no se paga hasta que alguien lo revise.

Cómo validar. Crear una orden por 10 piezas, recibir 9 y cargar la factura por 10: el sistema debe marcar la discrepancia y frenar el pago. Con orden, recepción y factura iguales, debe pasar sin fricción.

P0-4. Proveedores como catálogo, no como texto libre

Problema. Hoy el proveedor es un texto que cada quien escribe como quiere. «Oxxo», «OXXO» y «OXXO EXPRESS» quedan como tres proveedores distintos. Así no hay historial, no hay totales por proveedor y la detección de duplicados se debilita.

Qué implementar. Catálogo de proveedores por empresa, con RFC como llave. Al capturar, el sistema sugiere el proveedor existente que más se parece; crear uno nuevo requiere confirmación. La lectura automática guarda el texto original, pero el registro queda ligado al proveedor del catálogo.

Cómo validar. Capturar el mismo proveedor escrito de tres formas: las tres capturas deben terminar ligadas a un solo proveedor del catálogo, conservando el texto original de cada ticket.

P0-5. Políticas que se cumplen solas, no un botón de autorizar

Problema. Hoy autorizar es un botón. Un gasto de $50 y uno de $50,000 pasan por el mismo camino, sin tope, sin escalamiento y sin centro de costo obligatorio. Eso no es una política: es un clic.

Qué implementar.

- **Topes por concepto y por tipo de gasto:** cada concepto tiene un monto máximo sin autorización superior.
- **Escalamiento por monto:** entre más grande el gasto, más alto el nivel que debe aprobarlo. Los rangos los define cada empresa.
- **Centro de costo obligatorio:** sin centro de costo, el gasto no se registra. Sin ese dato no hay presupuesto que controlar después.
- **Registro de la decisión:** quién aprobó, cuándo y bajo qué regla. Ese registro entra a la cadena de custodia.

Cómo validar. Intentar registrar un gasto sin centro de costo: debe rechazarlo. Registrar un gasto por encima del tope del aprobador: debe escalarlo, no aprobarlo. Un gasto dentro de política debe pasar sin pasos extra.

P0-6. Revisar la ruta de tipos-gasto: posible error ya presente

Problema. La ruta de tipos-gasto usa el atributo `usuario.company_id` en repetidas ocasiones, pero el modelo de usuario (`UserModel`) no define ese atributo. Eso significa que esa ruta puede estar rota hoy, o a punto de romperse.

Qué implementar. Verificar primero si el atributo falta de verdad y en qué casos falla. La corrección natural llega con P0-1: si el usuario tendrá empresa y rol, el atributo debe existir en el modelo, con su migración. No se parcha la ruta a ciegas sin entender el modelo.

Cómo validar. Una prueba que ejerza la ruta de tipos-gasto con un usuario real debe pasar sin error de atributo, antes y después del cambio de P0-1.

P1 — Para que el cliente no se dé de baja a los 3 meses

P1 no bloquea la primera venta, pero decide si el cliente se queda. Se construye después de P0, en este orden sugerido:

Pieza	Qué falta y para qué sirve
Anticipos, viáticos y reembolsos	Hoy el dinero que un empleado gasta de su bolsa no tiene flujo. Se necesita: anticipo entregado, comprobación con tickets, diferencia a favor o en contra y reembolso, todo ligado y con aprobación.
Presupuesto por centro de costo	Presupuesto mensual por área, con lo gastado y lo comprometido en órdenes abiertas. Alerta al llegar al 80 % y aviso claro al agotarse. Sin esto, el centro de costo de P0-5 es solo una etiqueta.
Tarjetas y banca automática	Hoy el banco entra por CSV manual. Conectar tarjetas y cuentas para que los movimientos lleguen solos y la conciliación ticket–banco sea casi automática.
Notificaciones y cola del aprobador	El aprobador necesita una bandeja: qué le espera, desde cuándo y qué ya venció. Avisos por correo cuando algo requiere su decisión. Sin bandeja, las aprobaciones se pierden en mensajes.
Cierre con bloqueo y reapertura con motivo	Un mes cerrado no se toca. Si hay que reabrirlo, se exige motivo, queda registrado quién lo reabrió y el cierre nuevo deja rastro del anterior. La custodia ya te da la base.
Empleados como entidad	Hoy quien gasta es texto o un usuario del sistema. El empleado debe ser una entidad propia: nombre, área, centro de costo, jefe que aprueba. Es la base de viáticos, presupuestos y aprobaciones.

> **Criterio para subir algo de P1 a urgente:** si un cliente piloto lo pide por escrito y sin eso no renueva, se adelanta esa pieza sola. No se adelanta todo P1 por una sola petición.

P2 — Valor futuro, no arranque

Pieza	Qué es y cuándo tiene sentido
Captura móvil completa	Foto del ticket desde el celular, directa a la empresa correcta. Tiene sentido cuando ya hay empleados registrando gasto, no antes.
Multi-moneda real	Guardar moneda y tipo de cambio real de cada documento y reportar en pesos sin inventar conversiones. Crece desde lo que P0-2 deje guardado del CFDI.
Detección de anomalías	Avisos por patrones raros: mismo monto repetido, proveedor nuevo con monto alto, gasto fuera de horario o de política. Útil con volumen; con pocos datos solo genera ruido.
Webhooks e integración con ERP	Avisar a otros sistemas cuando un gasto se aprueba o un mes se cierra, y conectar con el sistema contable o ERP del cliente grande.
Respaldo automático y retención	Copias automáticas probadas y conservación de documentos por 5 años, como pide la obligación fiscal. El respaldo que nunca se ha probado restaurar no cuenta como respaldo.

Qué NO construir todavía

- **Una app por cliente.** Una sola app con separación por empresa. Una app por cliente es el caos de mantenimiento que quieres evitar cuando crezcan los clientes; solo se ofrece como opción premium y aislada si un cliente la paga.
- **Un chatbot o más inteligencia artificial por moda.** Nadie paga por eso en un cierre mensual.
- **Visión para todos los documentos.** Visión solo cuando OCR no alcanza: es la causa principal de la lentitud.
- **Cualquier P2 antes de cerrar P0.** Cada función nueva antes de separar empresas multiplica el trabajo de separación.

Escáner: rápido, preciso y confiable

La meta del escáner no cambió: **menos de 1 minuto por ticket y 85 % de precisión**. Son metas por medir, no resultados ya logrados. Así se ataca cada frente:

Rápido

- La foto HEIC se convierte **una sola vez** y esa misma imagen se reutiliza para OCR y para visión. Convertir dos veces es tiempo tirado.
- OCR prueba primero su mejor configuración, no las 16 lecturas a ciegas.
- OCR se paraleliza; visión sigue secuencial, porque es el recurso caro.
- El escaneo corre en segundo plano y devuelve un folio de corrida. Así el servidor web no corta a los 300 segundos y la pantalla no se queda en «cargando».

Preciso

- Validaciones sin llamar más modelos: total igual a subtotal más IVA más IEPS; RFC con formato válido; fecha válida y dentro del periodo; folio no repetido para el mismo proveedor.
- Esas mismas validaciones sirven de gatillo: si OCR entrega algo que no cuadra, ahí sí se llama a visión.

El plus que se escapaba: calidad de la foto

Antes de gastar OCR y visión, un chequeo rápido de la foto: ¿está borrosa?, ¿oscura?, ¿con reflejo o brillo?, ¿resolución muy baja? Si la foto no sirve, no se manda a visión: se marca **REVISAR_FOTO** y se le dice a la persona qué corregir —más luz, sin reflejo, ticket completo en cuadro— en vez de entregarle un total inventado después de minutos de espera.

Anti-duplicados: la regla y su estado real

La regla obligatoria

Antes de guardar cualquier ticket o factura, se valida si ya fue procesado **para la misma empresa**. Si ya existe, no se registra de nuevo, no suma a los totales y no entra al cierre, aunque la foto cambie de nombre o se vea distinta.

- **Tickets:** hash del archivo, más proveedor o RFC, fecha, total y folio. Si falta el folio: proveedor más fecha más total. Si falta la fecha: proveedor más total.
- **Facturas CFDI:** el UUID manda. Si ese UUID ya existe en la empresa, es duplicado.
- **Tres momentos de validación:** al subir el archivo, al terminar la lectura y antes de conciliar o cerrar.

El plus: listado de duplicados encontrados

Cada corrida y cada cierre entregan un resumen —cuántos se revisaron, cuántos son válidos y cuántos duplicados— y una fila por duplicado: documento, proveedor, fecha, total, folio o UUID, motivo, el registro original con el que choca y qué se hizo. Ese listado se exporta como anexo del informe en PDF y Excel.

Con tu set de prueba: Soriana $65.00 y Bodega Aurrera $234.00 aparecen dos veces cada uno. El resultado correcto es **5 válidos y 2 duplicados listados**, con la segunda copia rechazada y sin sumar.

> **Ya avanzado:** la validación por contenido está implementada en la rama `perf/scan-vision-solo-si-ocr-falla`, sin *commit*: se marca DUPLICADO y no se crea un segundo ticket ni se borra nada, y la respuesta del escaneo ya trae el listado de duplicados por contenido. Como en tu sistema no existe campo de folio, el listado lleva folio vacío hasta que ese campo exista; no se inventó.

> **Todavía no probado, dicho con claridad:** la corrida real con los 7 archivos HEIC aún no se ha hecho, porque los modelos de Ollama se estaban descargando. La precisión del 85 % —6 de 7 documentos correctos contra la verdad guardada— es la meta, **no un resultado**. Y la validación final con *ruff* y *pytest* de esta parte queda pendiente de correr en tu terminal limpia, sin variables viejas exportadas (ver Anexo B).

Orden recomendado: un paso a la vez

1. **P0-1 Roles y multi-empresa real** — el prompt del Anexo A. Nada empieza antes de que pase la prueba de fuga cruzada.
2. **P0-6 Tipos-gasto** — se verifica y se corrige junto con P0-1, porque depende del mismo atributo del usuario.
3. **P0-2 CFDI fiscal y validación SAT** — guardar lo que hoy se tira y dejar de inventar el tipo de cambio.
4. **Regla anti-duplicados extendida al CFDI por UUID** — sobre la base que ya existe para tickets.
5. **P0-4 Proveedores como catálogo** — primero el catálogo; la comparación triple lo necesita.
6. **P0-3 Comparación triple** — requisición, orden, recepción y factura.
7. **P0-5 Políticas ejecutables** — topes, escalamiento y centro de costo obligatorio.
8. **Medición del escáner con los 7 HEIC** — precisión contra la verdad y tiempo por ticket, cuando los modelos estén listos.
9. **P1 en orden de retención** — empleados, anticipos y viáticos, presupuesto, bandeja del aprobador, cierre con bloqueo, banca automática.
10. **P2 solo con clientes pagando** — móvil, multi-moneda real, anomalías, integraciones y respaldo probado.

> **Regla de oro para cada función nueva:** si no ayuda al cierre mensual ni al control del gasto, no entra. Cada función extra es tiempo de desarrollo, tiempo de soporte y una cosa más que puede fallar frente al cliente.

Cuándo sí pensar en una app por ambiente: cuando haya equipo de personas encargadas de operar, con planificación y respaldo probados. Hoy, con una sola app bien separada, mantienes a muchos clientes sin caos. Crecer de clientes no debe significar crecer de instalaciones.

Anexo A — Prompt ejecutable: P0-1, roles y multi-empresa

Copia todo el bloque siguiente y pégalo en una sesión nueva. No recortes las reglas: son las que evitan que el trabajo rompa lo que ya funciona.

```
CONTEXTO
Trabaja en expense-reconciler-mvp (FastAPI Python 3.11+, PostgreSQL 16
+ pgvector, SQLAlchemy 2.0 async, Pydantic v2, Docker Compose) en mi
MacBook Pro M3, en VS Code. Rama actual de trabajo si existe:
perf/scan-vision-solo-si-ocr-falla, base develop, SIN commit ni push
hasta que yo lo apruebe. Explicame cada comando en espanol sencillo,
linea por linea. No incluyas ninguna clave en tu respuesta.

ESTADO ACTUAL (no lo rehagas)
- Vision solo si OCR insuficiente; MAX_LADO_VISION=1000; logs
  vision_llamada/total_ms/vision_ms; tiempo total duracion_segundos
  en backend y front.
- Anti-duplicados por contenido ya implementado sin commit: si ya
  existe un ticket de la misma empresa con mismo proveedor + total
  + fecha (o proveedor + total si falta fecha), se marca DUPLICADO,
  no se crea un segundo ticket, no se borra nada, y ScanResponse trae
  duplicados_por_contenido. No existe campo folio: no lo inventes.
- Se conservan intactos: cadena de custodia, hash, append-only,
  muestra del 5%, veredicto Wilson y confidence_gate.

TAREA DE ESTA SESION: SOLO P0-1 (+ verificar P0-6)
1. Lee primero el codigo real: modelo de usuario, dependencias de
   autenticacion (deps), y como llega hoy company_id a cada ruta.
   Reporta lo que encuentres ANTES de cambiar nada.
2. La empresa se toma SIEMPRE del token del usuario autenticado.
   PROHIBIDO tomarla de un parametro, del cuerpo o de una cabecera
   que el cliente controle. Toda consulta filtra por esa empresa.
3. Agrega al usuario empresa y rol (admin, contador, aprobador,
   digitalizador, solo_lectura) con su migracion. Cada rol solo
   puede hacer lo suyo; lo demas responde sin permiso.
4. Activa Row Level Security en PostgreSQL como segunda barrera.
5. Empresa demo separada: ningun dato demo se mezcla con reales.
6. P0-6: verifica si tipos-gasto usa usuario.company_id sin que
   UserModel lo defina; reportalo y dejalo corregido por el paso 3.

VALIDACION OBLIGATORIA (sin esto, no esta terminado)
- Prueba de fuga cruzada: usuario de empresa A pide por su ID exacto
  un ticket, factura, cierre y reporte de empresa B: los 4 deben dar
  no encontrado o sin permiso, NUNCA datos de B.
- Tests de roles: cada rol intenta una accion que no le toca y falla.
- ruff y pytest completos en verde. Si fallan, dime cual, por que y
  que propones; no ocultes ningun fallo.
- No prometas precision de escaner: el 85% es meta por medir con los
  7 HEIC, no un resultado.

ENTREGA
Resumen en espanol sencillo: archivos tocados, que cambio en cada
uno, resultado de la prueba de fuga y de ruff/pytest, y que sigue
pendiente. Si te quedas sin tokens, cierra con un resumen de donde
quedamos y un prompt corto para continuar donde ibas.
```

Anexo B — Comandos explicados y cómo continuar

Estos comandos se corren en tu Mac, en la terminal de VS Code, dentro de la carpeta del proyecto. Cada uno se explica en español sencillo.

```
ruff check app/services/scan_service.py app/schemas/scan.py app/api/scans.py
tests/unit/test_scan_service.py
```

Qué hace: revisa que el código de la regla anti-duplicados no tenga errores de estilo ni importaciones sin uso. No cambia nada, solo revisa. Debe terminar con «All checks passed».

```
PYTHONPATH=. pytest tests/unit/test_scan_service.py tests/unit/test_capture.py -q
```

Qué hace, por partes: `PYTHONPATH=.` le dice a Python que busque el código en la carpeta actual. `pytest` corre las pruebas automáticas de esos dos archivos. `-q` muestra el resultado corto. Si tu terminal trae variables viejas exportadas de sesiones anteriores, las pruebas pueden fallar al leer la configuración: ciérrala, abre una nueva y vuelve a intentarlo antes de tocar código.

```
docker exec expense_ollama du -sh /root/.ollama/models
```

Qué hace: entra al contenedor de Ollama y muestra cuánto espacio ocupan los modelos ya descargados. El total esperado es cercano a 5.2 G entre los dos modelos. Sirve solo para saber cuánto falta; no acelera ni cancela la descarga.

Si se acaban los tokens a mitad del desarrollo

Pega este prompt corto en una sesión nueva:

```
Continua P0-1 en expense-reconciler-mvp: empresa siempre desde el
token, nunca desde el cliente; roles admin/contador/aprobador/
digitalizador/solo_lectura; Row Level Security; prueba de fuga
cruzada entre empresas obligatoria; conserva custodia, hash,
confidence_gate y anti-duplicados ya implementados sin commit.
Explicame los comandos linea por linea y no des por bueno nada
que ruff y pytest no hayan pasado.
```

> **Lo que este documento deja cerrado:** el diagnóstico, el orden y el primer paso listo para ejecutar. Lo que no deja cerrado —y nadie debe dar por hecho— es el código de P0-1 y la medición del escáner: ambos se prueban, no se declaran.